

The Executable Charter: Architecture

Terms that run as code. Every rupee has a receipt.

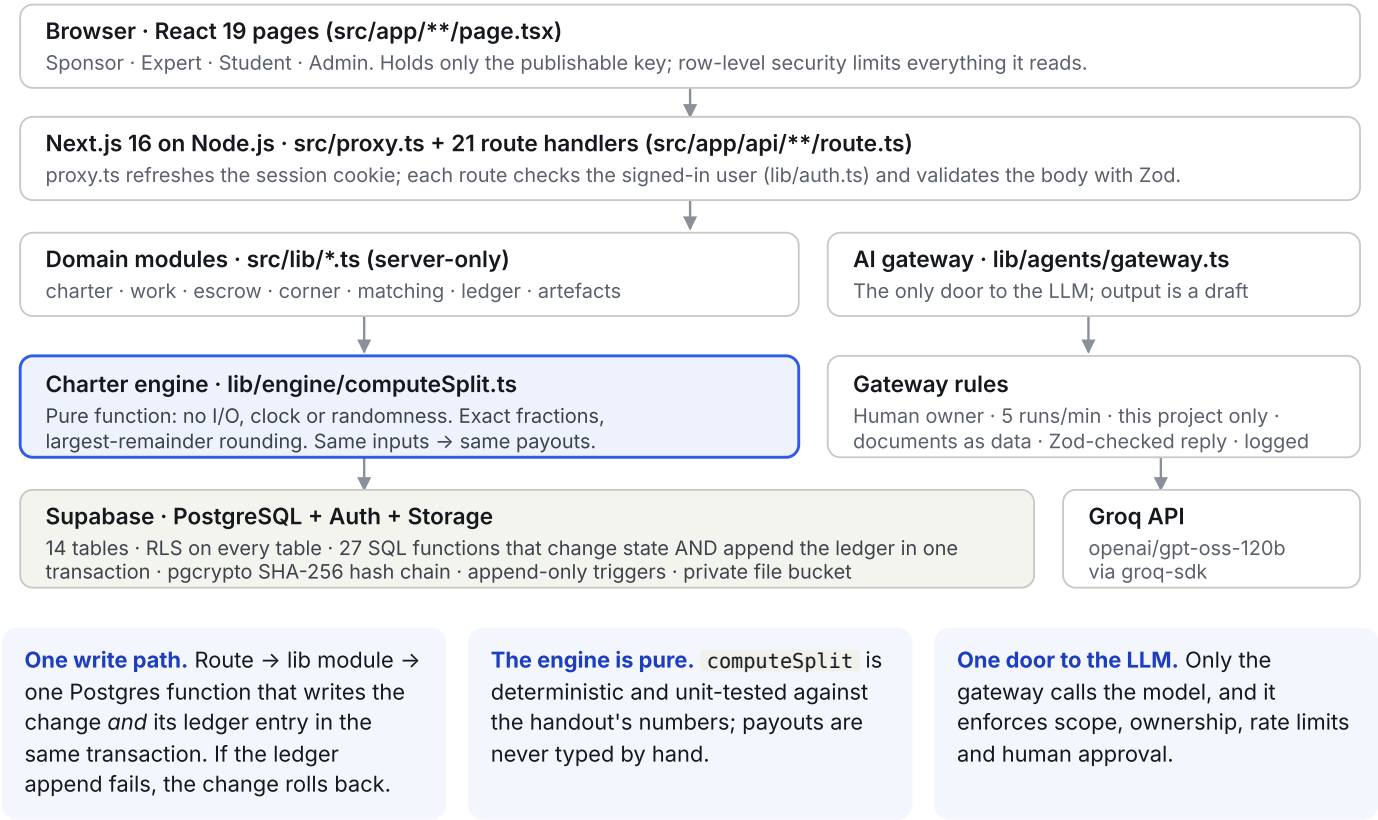
Sponsors post research problems; experts and students form verified teams and work alongside AI agents. Our core idea: the **project charter is structured data that a pure engine runs**, not a PDF. The engine reads the charter plus a **tamper-evident, hash-chained ledger** of reviewed contributions, and produces every payout with a plain-English **receipt**. AI agents only ever produce **drafts** that a named human approves.

Who did what?
Every human and AI action is one append-only, hash-chained ledger entry with a named human owner.

Who could see what?
Postgres row-level security locks the confidential brief until the exact charter version is accepted. Every view is logged.

Why paid what?
One pure function runs the charter on expert-reviewed work and stores a receipt for every person.

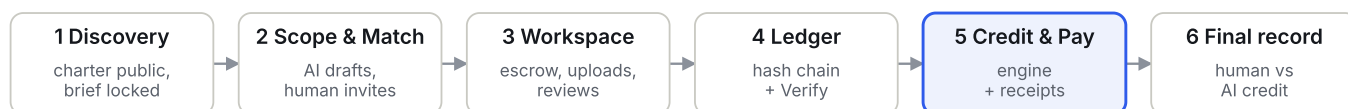
1 System architecture



Tech stack

LAYER	CHOICE	WHY
App + API	Next.js 16.3 (App Router) on Node.js 22, React 19, TypeScript 5, Tailwind CSS 4	One language and repo; server-side rendering keeps secrets off the browser
Data, auth, files	Supabase: PostgreSQL, Auth, Storage (supabase-js 2.117, @supabase/ssr 0.12)	Real transactions, triggers, RLS and pgcrypto for the ledger
AI	Groq (groq-sdk 1.6), model openai/gpt-oss-120b from GROQ_MODEL	Fast, low-cost hosted open-weight model; swappable by one variable
Validation, tests	Zod 4.6 · Vitest 5 (78 tests) · PGlite 0.5.8 (in-process Postgres) · ESLint 9	Validate every request and AI reply; test real SQL locally

2 The journey: Contribution → Verification → Attribution → Credit / Payment



Each step is one page under `/projects/[id]`. Every button calls one SQL function, e.g. `accept_charter` → `CHARTER_ACCEPTED`, `fund_escrow` → `ESCROW_FUNDED`, `add_contribution` → `CONTRIBUTION_ADDED`, `add_review` → `REVIEW_DONE`, `accept_milestone` → `PAYOUT_ISSUED`.

3 Data model (14 tables)

TABLE	KEY FIELDS AND PURPOSE
<code>profiles</code>	role (student / expert / sponsor / admin), verified, skills, proven skills, hours per week, affiliations, minor + guardian consent
<code>projects</code> · <code>project_briefs</code>	sponsor, public summary, sensitivity, declared competitors · the confidential brief, in its own table so a policy can lock it
<code>charters</code> · <code>memberships</code>	versioned terms (engagement model, fee, AI reserve, expert %, split rule, IP) · status (invited / active / exited), accepted charter version , active fraction
<code>milestones</code> · <code>escrows</code>	acceptance criteria, amount, status · funded → released / frozen / refunded (simulated money)
<code>tasks</code> · <code>contributions</code> · <code>reviews</code>	one human owner per task · human owner, agent used, AI share, artefact SHA-256, similarity, version + builds_on · verdict, impact 0–10
<code>agent_drafts</code> · <code>payouts</code> · <code>disputes</code>	AI output awaiting approval · amount + stored receipt lines + charter version used · reason, cited criterion, resolution
<code>ledger</code>	seq, timestamp, project, actor, on_behalf_of , event, payload (JSON), prev_hash, hash

4 Ledger: tamper evidence



hash = SHA-256(prev_hash | seq | timestamp | actor | on_behalf_of | event | payload), computed in SQL by `ledger_hash()`; first prev_hash = 64 zeros.

PROPERTY	HOW IT IS ENFORCED
Linear under concurrency	<code>ledger_append_entry</code> takes a transaction-level advisory lock, so two writes can never share a previous hash
Append-only	Triggers <code>ledger_append_only</code> and <code>ledger_no_truncate</code> block UPDATE, DELETE and TRUNCATE; a mistake is fixed with a CORRECTION entry
No unowned AI action	The append refuses an agent entry without <code>on_behalf_of</code> (the responsible human)
Verifiable	<code>ledger_verify()</code> re-hashes from #1 and returns ok, the first broken seq, and the count checked; append and verify share one SQL implementation
Why not a blockchain	One trusted operator needs tamper <i>evidence</i> , not consensus: no gas, no wallets, instant writes. Production would anchor the latest hash publicly

5 Charter engine and reward logic

The charter is published before anyone applies, accepted **per version**, and never changed silently: a new version re-locks the brief until each member re-accepts, and anyone who disagrees can leave with their credit. All percentages below are charter fields.

```
distributable = budget - fee 10% - AI reserve 5%
expert total  = 30% × distributable
student pool  = distributable - expert total
student_i     = 40% × pool ÷ n × activeFraction_i
               + 60% × pool × weight_i
weight_i      = reviewed impact_i ÷ Σ reviewed impact
```

Exact fractions with `BigInt ( fraction.ts )`, never floats. Rounding to whole rupees by largest remainder, ties to the smallest share, so payouts add up exactly. Weights come from expert reviews, never commit counts. Only people on the team when the milestone was submitted share in it.

₹1,00,000 MILESTONE	EQUAL	WEIGHTED	TOTAL
Dr. Kiran (expert)	—	—	₹25,500
Priya (impact 5)	₹7,933	₹17,850	₹25,783
Arjun (impact 3, 70% AI)	₹7,933	₹10,710	₹18,643
Meera (impact 2)	₹7,933	₹7,140	₹15,074
Paid to people			₹85,000

+ ₹10,000 fee + ₹5,000 AI reserve = ₹1,00,000 exactly. A **knowledge-sharing** project runs the same engine with ₹0: acceptance issues credentials and co-author credit instead of money.

6 Access control: who could see what

DATA	SPONSOR	ACTIVE MEMBER	ADMIN	OUTSIDER	AI AGENT
Summary + charter	Yes	Yes	Yes	Yes	This project only
Confidential brief	Yes	After accepting the latest version	No	No (403)	Its owner's project only
Documents, reviews, ledger	Yes	Yes	Yes	No	Review agent only
Payouts + receipts	Yes	Yes (whole team)	Yes	No	Never
Approve, review, pay	Accept, pay	Own work; expert reviews	Disputes	No	Never

Enforced by Postgres row-level security (default deny; helper functions `can_read_brief`, `is_project_member`, `is_project_sponsor`), not by the page. Clients cannot write any table directly. Every brief view is logged as `BRIEF_VIEWED`; access ends the moment a member exits. The secret key exists only on the server.

7 AI agents: drafts under a human owner

AGENT	OWNER	CAN READ	OUTPUT (ALWAYS A DRAFT) → HUMAN DECISION
Scoping	Sponsor	Summary + brief	Milestones, skills, acceptance criteria → sponsor approves the scope
Matching explainer	Sponsor / admin	Score breakdown only	One-line reason per candidate → a human sends invites
Research / Coding	The student	This project's summary, brief, own notes	Draft note or code → owner edits and approves it as their contribution
Review	Expert	Criteria + submission record	Criteria check (met / not met / unclear) → the expert's own review and impact score

Gateway (`runAgent`): owner must be the sponsor or an active member on the current charter · max 5 runs per user per minute · documents wrapped in `<document>` tags as data, never instructions (prompt-injection defence, unit-tested) · reply validated by Zod · logged as `AGENT_ACTION` with owner, tokens and cost · AI compute paid from the 5% reserve; credit stays with the human and the AI share is printed on the receipt.

Matching: hard filters (verified, guardian consent for minors on paid work, no conflict of interest with declared competitors, enough hours, < 2 active projects, ≥ 1 required skill) → score /100 (skills 40, track record 20, topic fit 15, availability 15, newcomer boost 10) → the LLM only explains → top 5 per role → a human invites → the candidate accepts the charter.

8 Corner cases

CASE	WHAT HAPPENS	MECHANISM
Student quits midway	Reviewed credit kept; equal share pro-rata (Meera at 60%: ₹15,074 → ₹11,900, ₹3,173 unallocated); access revoked at once	<code>exit_member</code> , <code>activeFraction</code>
Sponsor goes silent	Auto-accept after the charter's review window (10 days); the engine pays out	<code>charter term</code> , <code>accept_milestone</code>
Paid becomes unpaid	New charter version; briefs re-lock; unreleased escrow refunded; paid money stays paid; members re-accept or leave with credit	<code>publish_charter_version</code>
Unfair rejection	Rejection must cite a criterion; team disputes; escrow frozen until an admin decides	<code>reject_milestone</code> , <code>raise_dispute</code> , <code>resolve_dispute</code>
AI did most of the work	Credit to the human owner; AI share disclosed on ledger and receipt; AI earns nothing	<code>gateway</code> , <code>on_behalf_of</code>
Copied work	≥ 80% similarity flagged for an expert, never auto-blocked; original resubmitted as v2	SHA-256 + similarity + review
Late joiner · two editors	Shares only milestones submitted after joining · versions are append-only (<code>builds_on</code>), one owner per task	<code>split-team rule</code> , <code>tasks</code>

The first four are triggered live from the admin console; each calls the same SQL function a real user would and is marked `demo_control_by` in the ledger.

9 Security per threat

THREAT	MECHANISM	PROTOTYPE
Plagiarism / undisclosed AI	Mandatory AI-use declaration, similarity flag + human review, agent actions auto-logged	Score mocked
Insider leaks the brief	Brief only after accepting the charter, every view logged, access ends on exit	Real
Sponsor takes work, won't pay	Escrow funded before work, ledger timestamps as evidence, rejection must cite a criterion	Escrow simulated
Credit theft	SHA-256 of each artefact at submission + tamper-evident ledger; files stored privately by hash	Real
Fake accounts / conflicts	Verified flag, conflict-of-interest filter, human invitation	KYC simulated
AI data leak / prompt injection	Project-scoped gateway, documents as data, human approval	Real
Bait-and-switch terms	Versioned charter, re-acceptance, accepted version recorded	Real
Ledger / payment tampering	Append-only hash chain, corrections as new entries, Verify	Real

10 Simplifications

HERE	PRODUCTION
Simulated escrow	Licensed escrow / payment partner
Simulated KYC	Government-ID verification
Mock similarity	Real plagiarism + AI-detection service
Exits / silence triggered by admin	Scheduled jobs (nudge day 5, inactive day 7)
One ledger chain	Per-project chains, hash anchored publicly

Not built: per-viewer watermarking of the brief. All users and data are synthetic.

11 Testing and changes since Round 2

78 Vitest tests: the engine against the handout's numbers (plus pro-rata exit and ₹0 budget); schema, RLS, hash chain, tamper detection and every state-changing SQL function on **PGlite**; access, matching, integrity and the prompt-injection defence. `npm run demo:dryrun` runs ~60 live HTTP checks against Supabase and found 3 bugs, all fixed.

Changed since Round 2: hashing moved into SQL so change + ledger commit together · exact-fraction engine · model is `openai/gpt-oss-120b` on Groq (no Llama chat model on our key; one variable to change) · payouts transparent inside the team · split-team rule · tasks and versions added · uploads stored privately by hash.